Abstract Machines

1 Introduction

2 Abstract C Machine

3 Prolog: Warren Abstract Machine

Abstract Machines

Abstract machines are used in theoretical computer science to study the (theoretical) properties of computation machinery. Examples:

Turing machine	Computability
RAM	Complexity of sequential computations
PRAM	Complexity of parallel computations

In the area of programming languages abstract machines are also used for the specification of the semantics of a language. Often, efficiency is not important then, e.g., for the abstract C machine. For some languages, abstract machines have been created explicitly to support an efficient implementation of the language, e.g., the Spineless Tagless G-Machine for Haskell and the Warren Abstract Machine for Prolog.

Abstract Machines

1 Introduction

2 Abstract C Machine

3 Prolog: Warren Abstract Machine

Spezification of the Semantics of C

The C99 standard (ISO/IEC 9899:1999) specifies the behavior of C programs in terms of an abstract machine.

“The semantic descriptions in this International Standard describe the behavior of an abstract machine in which issues of optimization are irrelevant. [...] In the abstract machine, all expressions are evaluated as specified by the semantics. An actual implementation need not evaluate part of an expression if it can deduce that its value is not used and that no needed side effects are produced (including any caused by calling a function or accessing a volatile object). [...]”
(ISO/IEC 9899:1999, Section 5.1.2.3)

Undefined Behavior in C

The abstract C machine also specifies, in which cases the behavior is undefined.
Example: `i = i++;`

Here, it is only specified that the value of `i` is increased (due to `i++`) after the value of `i` has been read (on the right-hand side). It is not specified whether the effect of `i++` or the assignment are executed first.

Not all C compilers are capable of warning the programmer when undefined behavior is used; sometimes, compilers even try to implement “expected” behavior.

Overflows

Overflows with interger types in C are only defined for unsigned types. For signed types overflows are *undefined* (in contrast to Java). In the implementation, signed types have overflow behavior *most of the time* because the machine instructions used to implement them have it.

Overflows

Overflows with interger types in C are only defined for unsigned types. For signed types overflows are *undefined* (in contrast to Java). In the implementation, signed types have overflow behavior *most of the time* because the machine instructions used to implement them have it.

The optimizer of the compiler can exploit undefinedness in the optimizations performed. That is why the following test for overflow in $s+1$ does *not* work:

```
int s;
...
if (s+1 < s)    /* s+1 is an overflow */
abort();
char *str = malloc(s+1);
```

Without overflow, `s+1 < s` is false all the time; therefore, the compiler can simplify the conditional to `false`!

Another Example

According to the standard, dereferencing `NULL` pointers is undefined.

```
dead = *p;
if (p == NULL) { return; }
*p = 42;
```

What can the compiler do with this code?

Another Example

According to the standard, dereferencing `NULL` pointers is undefined.

```
dead = *p;
if (p == NULL) { return; }
*p = 42;
```

What can the compiler do with this code?

- Since `dead` is not used, `dead = *p` is removed by *dead code elimination*. The test `p == NULL` is not superfluous (and remains in the code).
- Since `*p` occurs before `p == NULL`, the code piece is only meaningful if `p != NULL`; therefore, the `if` clase can be removed entirely.

Typical result: with optimizations, the code performs the check `p != NULL` (due to dead code elimination), without optimizations it does not.

C and Parallelism

Older C standards (before C 2011) specified the behavior of a C program without concurrency.

- Write accesses to memory can be arranged freely by the compiler (as long as data dependences in the considered code are not violated).
- When using thread libraries (`pthreads`, etc.) and C11, one has to pay attention to use synchronizations when accessing shared data, e.g., using the C11 functions `atomic_thread_fence`.
- Language extensions (e.g. `OpenMP`) provide explicit constructs for memory synchronization (such as the `flush` directive in `OpenMP`). Example:

```
while (!flag) { ... #pragma omp flush(flag) }

// other thread
flag = true;
#pragma omp flush(flag)
```

Abstract Machines

1 Introduction

2 Abstract C Machine

3 Prolog: Warren Abstract Machine

Prolog

Prolog is a declarative programming language based on relations (logic programming). The declarative semantics of Prolog programs[1] can be defined quite easily. The question of how to implement Prolog efficiently / for an operational semantics is answered by the *Warren Abstract Machine*.

Warren Abstract Machine

The *Warren Abstract Machine* (D. Warren, 1983) is a low-level, operational description of the execution of Prolog programs.

[1] without special constructs

Prolog

A Prolog program consists of *clauses*. Clauses are statements over the *relations* between *terms*. A relation is written in the form $r(t_1, \ldots, t_n)$ where r is the name of the relation and $t_1, \ldots, t_n$ are the argument terms of the relation. Clauses can be:

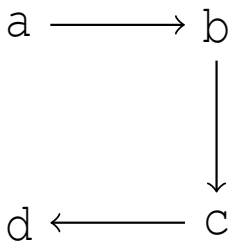
facts in the form $r(t_1, \ldots, t_n)$.
"$r(t_1, \ldots, t_n)$ holds."

rules in the form $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots)$.
"$r(t_1, \ldots, t_n)$ holds, if $p_1(\ldots), \ldots, p_m(\ldots)$ hold."
$p_1, \ldots, p_n$ are called *goals*.

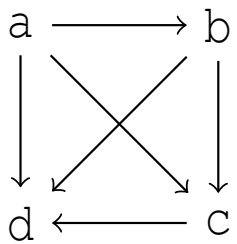
Terms are either variables (starting with a capital letter), constants or compound terms $t(t_1, \ldots, t_n)$.

Example

```
edge(a,b).
edge(b,c).
edge(c,d).
```

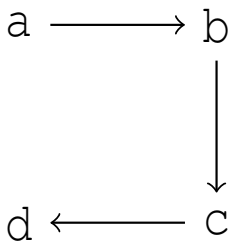



```
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```

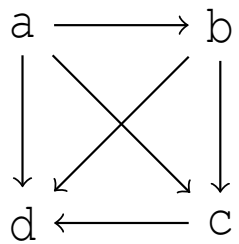


Example

```
edge(a,b).
edge(b,c).
edge(c,d).
```




```
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```

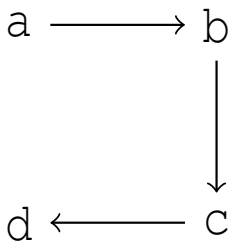


We can pose *queries* to the progam, for example:

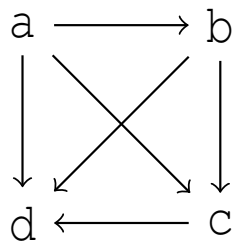
- `?- path(a,c).` “Is there a path from `a` to `c`?”

Example

```
edge(a,b).
edge(b,c).
edge(c,d).
```




```
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```

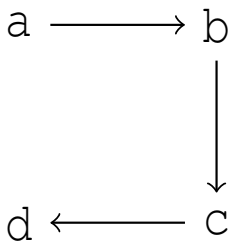


We can pose *queries* to the progam, for example:

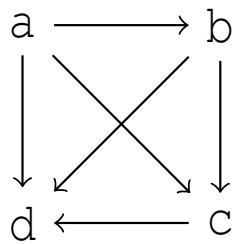
- `?- path(a,c).` “Is there a path from `a` to `c`?”
  answer: `true`

Example

```
edge(a,b).
edge(b,c).
edge(c,d).
```




```
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```

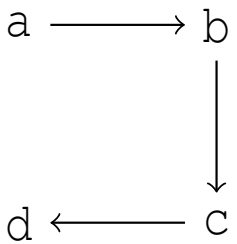


We can pose *queries* to the progam, for example:

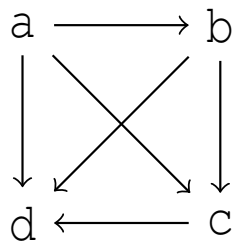
- `?- path(a,c).` “Is there a path from `a` to `c`?”
  answer: `true`
- `?- path(S,c).` “From where can one get to `c`?”

Example

```
edge(a,b).
edge(b,c).
edge(c,d).
```




```
path(X,Y) :- edge(X,Y).
path(X,Y) :- edge(X,Z), path(Z,Y).
```



We can pose *queries* to the progam, for example:

- `?- path(a,c).` “Is there a path from `a` to `c`?”
  answer: `true`
- `?- path(S,c).` “From where can one get to `c`?”
  answer: `S=b`, `S=a`

Processing of Queries

A query q is processed by looking in the program for a clause $r(\ldots) :- p_1(\ldots), \ldots$ such that $r(\ldots)$ matches a goal $r(\ldots)$ in q. In q, $r(\ldots)$ is replaced by the right-hand side of the clasue $(p_1(\ldots), \ldots)$. In this process variables are bound to terms, i.e., an environment (mapping variables to values) is constructed. This process repeats until q is empty. The constructed variable bindings represent the answer to the query. When no clause matches for a non-empty q, there is no answer.

Example:

`path(S,c)`	rule `path(X,Y):- edge(X,Z),path(Z,Y).`
$\rightarrow$ `edge(S,Z),path(Z,c)`	fact `edge(a,b).` with `S=a,Z=b`
$\rightarrow$ `path(b,c)`	rule `path(X,Y) :- edge(X,Y).`
$\rightarrow$ `edge(b,c)`	fact `edge(b,c).`
$\rightarrow$ `.`	answer: `S=a`

In general, there can be several answers to a query, even infinitely many are possible.

Several Answers

```
   path(S,c)
→ edge(S,Z),path(Z,c) with S=a,Z=b
→ path(b,c)
→ edge(b,c)
→ .
```

or:

```
→ edge(S,c)
→ .          with S=b
```

For a goal *g* in the query, *all* heads of clauses have to be tried. For space reasons, one performs a depth-first search (instead of a breadth-first search), i.e., one returns to an earlier location and tries the next clause as soon as processing the chosen alternative is complete. When returning to an earlier location, variable bindings have to be restored (to undo the effect of the now finished alternative).

Interpretation of Prolog

In principle, a primitive interpreter for Prolog can be implemented using a stack containing all the still to be processed goals. When a head of a clause matches the goal on top of stack, it is replaced by the right-hand side of the clause and the variable bindings are updated. When several clauses match, the stack has to be duplicated such that one can continue for every alternative.

Interpretation of Prolog

In principle, a primitive interpreter for Prolog can be implemented using a stack containing all the still to be processed goals. When a head of a clause matches the goal on top of stack, it is replaced by the right-hand side of the clause and the variable bindings are updated. When several clauses match, the stack has to be duplicated such that one can continue for every alternative.

This is not an efficient implemtation. The questions arising are:

- How can one do without duplicating the stack?
- When an alternative is finished, to which state does one return and which variable bindings have to be reset?

Compilation of Prolog

Clauses are implemented as *code*, goals are calls to the corresponding code. The parameter passing is performed (conceptually) by reference. We use JAVA in the following examples.

Example: `path(X,Y) :- edge(X,Z), path(Z,Y).`

First try:

```
void path(Term X, Term Y) {
    Var Z = new Var();
    edge(X,Z);
    path(Z,Y);
}
```

Compilation of Prolog

Clauses are implemented as *code*, goals are calls to the corresponding code. The parameter passing is performed (conceptually) by reference. We use JAVA in the following examples.

Example: `path(X,Y) :- edge(X,Z), path(Z,Y).`

First try:

```
void path(Term X, Term Y) {
    Var Z = new Var();
    edge(X,Z);
    path(Z,Y);
}
```

↯ When `edge` fails, `path` must not be called.

Compilation of Prolog

Clauses are implemented as *code*, goals are calls to the corresponding code. The parameter passing is performed (conceptually) by reference. We use JAVA in the following examples.

Example: `path(X,Y) :- edge(X,Z), path(Z,Y).`

First try:

```
void path(Term X, Term Y) {
    Var Z = new Var();
    edge(X,Z);
    path(Z,Y);
}
```

↯ When `edge` fails, `path` must not be called.

Second try:

```
boolean path(Term X, Term Y) {
    Var Z = new Var();
    return edge(X,Z) && path(Z,Y);
}
```

Compilation of Prolog

Clauses are implemented as *code*, goals are calls to the corresponding code. The parameter passing is performed (conceptually) by reference. We use JAVA in the following examples.

Example: `path(X,Y) :- edge(X,Z), path(Z,Y).`

First try:

```
void path(Term X, Term Y) {
    Var Z = new Var();
    edge(X,Z);
    path(Z,Y);
}
```

↯ When `edge` fails, `path` must not be called.

Second try:

```
boolean path(Term X, Term Y) {
    Var Z = new Var();
    return edge(X,Z) && path(Z,Y);
}
```

↯ Works only if there is at most one answer.

Continuations

To be able to output anwsers for successful queries *and* to implement the choice between alternatives, one uses *continuations*. A continutation represents the remaining computation. In our case of compiling Prolog, a continuation is called in case of success. In JAVA, we can use a Lambda expression to pass continuations to functions.

```
void path(Continuation cont, Term X, Term Y) {
    Var Z = new Var();
    Continuation cont_path = () -> path(cont, Z, Y);
    edge(cont_path, X, Z);
}
```

`edge` calls, when it has succeeded, `cont_path`. `path` then calls, when it has also succeeded, `cont`, etc.

(For a successful termination, this example lacks the other alternative for `path`.)

Queries

A query is compiled just as the right-hand side of a clause. The initial continuation (passed to the last goal of the query) outputs the variable bindings.

```
Term S = new Var();
Term c = new Str("c");

Continuation done = () -> {
    System.out.println("Solution: S=" + S);
};

path(done, S, c);
```

Alternatives

Alternatives call, because they are conceptually on the same level, the same continuation.

```
void path1(Continuation cont, Term X, Term Y) {
    edge(cont, X, Y);
}

void path2(Continuation cont, Term X, Term Y) {
    Term Z = new Var();
    Continuation cont_path = () -> path(cont, Z, Y);
    edge(cont_path, X, Z);
}

void path(Continuation cont, Term X, Term Y) {
    path1(cont, X, Y);
    path2(cont, X, Y);
}
```

Restoration of Bindings

When there are two or more alternatives, there must be (starting from the second alternative) a restoration of the original situation w.r.t. the variable bindings before choosing the next alternative. The information about variable bindings made when an alternative is processed is stored in the so-called *trail*. The trail is a stack that grows by one element when a relation with alternatives is entered.

```
void path(Continuation cont, Term X, Term Y) {
    Var.pushTrail();    /* (*) new entry on trail */
    path1(cont, X, Y);
    Var.restoreVars();  /* restore variables as they were at (*) */
    path2(cont, X, Y);
    Var.restoreVars();  /* restore variables as they were at (*) */
    Var.popTrail();     /* remove entry from trail */
}
```

Variable Bindings

The binding of variables is performed using unification, i.e., two terms (which can be variables themselves or contain variables) are “matched” syntactically. This happens in `unify(...)`.

```
void edge(Continuation cont, Term X, Term Y) {
    Term a = new Str("a");
    Term b = new Str("b");
    Term c = new Str("c");
    Term d = new Str("d");
    Var.pushTrail();
    if (unify(a, X) && unify(b, Y)) { cont.call(); }
    Var.restoreVars();
    if (unify(b, X) && unify(c, Y)) { cont.call(); }
    Var.restoreVars();
    if (unify(c, X) && unify(d, Y)) { cont.call(); }
    Var.restoreVars();
    Var.popTrail();
}
```

Warren Abstract Machine

The Warren Abstract Machine realizes the principles sketched on the previous slides for the compilation of Prolog *without* using high-level programming language features such as local functions. The machine manages several stack structures itself:

- *heap* (also called the “global stack”) for terms (variables and function terms),
- (local) *stack* for local variables, used to keep track of the still to be processed alternatives and saving registers,
- *trail* for the restoration of variable bindings.

The machine uses registers $X_1, X_2, \ldots$ for local values and parameter passing; for argument registers, the names $A_1, A_2, \ldots$ are used but these are just other names for $X_1, X_2, \ldots$. The register S points to the next (sub-)term to be processed on the heap.

Slots in the local stack for values are called $Y_1, Y_2, \ldots$.

Representation of Terms

Terms are represented on the heap or the local stack as a pair (t, a). t is a *tag* and can take two possible values:

- REF: (t, a) represents a variable. a is the address of the term to which the variable is bound. When a points to (t, a) itself, the variable is unbound.
- STR: (t, a) represents a term; the constructor and the arguments of the constructor are stored consecutively starting at address a.

Example: `cons(X,cons(X,Y))`

1	2	3	...	10	11	12	...	20	21	22
REF,1	REF,2	STR,10	...	cons	REF,1	STR,20	...	cons	REF,1	REF,2

The registers contain references to the addresses in the heap or the local stack, respectively.

Overview of the WAM Instructions

V_i is either X_i or Y_i.

`get_variable` V_i,A_j	copy A_j to V_i
`get_value` V_i,A_j	unify V_i and A_j
`put_variable` V_i,A_j	save reference to new variable in V_i and A_j
`put_value` V_i,A_j	copy V_i to A_j
`get_structure` f,A_i	match A_i against f and set S
`put_structure` f,A_i	put f on the heap and save reference to it in A_i
`set_variable` V_i	create new variable on the heap and save it to V_i
`set_value` V_i	save V_i to heap
`unify_variable` V_i	load V_i from address S; $S := S+1$
`unify_value` V_i	unify V_i and the term at address S; $S := S+1$
`call` p	call routine p, set next instruction as continuation
`proceed`	call continuation
`allocate` n	allocate n slots $(Y_1, \ldots, Y_n)$ on the stack
`deallocate`	remove local variables and execute `proceed`
`try_me_else`	initialize trail for current function
`retry_me_else`	reset variable bindings
`trust_me`	reset variable bindings and reduce trail

...

Manipulation of Variables

- `get_variable` X_i,A_j or `get_variable` Y_i,A_j:
  copy A_j to X_i or Y_i
- `get_value` X_i,A_j or `get_value` Y_i,A_j:
  unify X_i or Y_i with A_j, i.e., variables in subterms of X_i or Y_i and A_j are bound to the corresponding subterms
- `put_variable` X_i,A_j or `put_variable` Y_i,A_j:
  create new variable in the heap and store reference to it in X_i and A_j or create it in the stack (in Y_i) and store reference to it in A_j
- `put_value` X_i,A_j or `put_value` Y_i,A_j:
  copy X_i or Y_i to A_j

Manipulation of Terms

- `put_structure` f,A_i:
  create a STR cell and a cell with f (to which the STR cell refers) on the heap and save reference to the new STR cell it in A_i
- `set_variable` X_i or `set_variable` Y_i:
  create new variable on the heap and save referenc to in in X_i or on the stack in Y_i
- `set_value` X_i or `set_value` Y_i:
  allocate new cell on the heap and store X_i or Y_i to it
- `get_structure` f,A_i:
  check if the term referenced by A_i can be unified with $f(\ldots)$ and set S to the first argument of the f term
- `unify_variable` X_i or `unify_variable` Y_i:
  let X_i or Y_i point to the term at address S and increment S (such that it points to the next constructor argument in the term)
- `unify_value` X_i or `unify_value` Y_i:
  unify the term at address S with X_i bzw Y_i and increment S (such that it points to the next constructor argument in the term)

Compilation of Facts, Rules and Queries

The code for a rule $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots).$
first checks if the arguments passed to it (in $A_1, \ldots, A_n$) match $t_1, \ldots, t_n$.
Then, $p_1, \ldots, p_m$ are called; for each call, the arguments of the respective p_i are loaded into $A_1, \ldots$. Facts are the special case with $m = 0$ and queries the special case with $n = 0$.

Local variables and arguments which are not only used in p_1 have to be saved in the stack (in $Y_1, \ldots$) for later use.

Compilation of Facts, Rules and Queries

The code for a rule $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots).$
first checks if the arguments passed to it (in $A_1, \ldots, A_n$) match $t_1, \ldots, t_n$.
Then, $p_1, \ldots, p_m$ are called; for each call, the arguments of the respective p_i are loaded into $A_1, \ldots$. Facts are the special case with $m = 0$ and queries the special case with $n = 0$.

Local variables and arguments which are not only used in p_1 have to be saved in the stack (in $Y_1, \ldots$) for later use.

Example: `path(K,L) :- edge(K,M), path(M,L).`

Compilation of Facts, Rules and Queries

The code for a rule $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots).$
first checks if the arguments passed to it (in $A_1, \ldots, A_n$) match $t_1, \ldots, t_n$.
Then, $p_1, \ldots, p_m$ are called; for each call, the arguments of the respective p_i are loaded into $A_1, \ldots$. Facts are the special case with $m = 0$ and queries the special case with $n = 0$.

Local variables and arguments which are not only used in p_1 have to be saved in the stack (in $Y_1, \ldots$) for later use.

Example: `path(K,L) :- edge(K,M), path(M,L).`

```
path:
  allocate 2                % 2 local variables: Y1, Y2
```

Compilation of Facts, Rules and Queries

The code for a rule $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots).$
first checks if the arguments passed to it (in $A_1, \ldots, A_n$) match $t_1, \ldots, t_n$.
Then, $p_1, \ldots, p_m$ are called; for each call, the arguments of the respective p_i are loaded into $A_1, \ldots$. Facts are the special case with $m = 0$ and queries the special case with $n = 0$.

Local variables and arguments which are not only used in p_1 have to be saved in the stack (in $Y_1, \ldots$) for later use.

Example: `path(K,L) :- edge(K,M), path(M,L).`

```
path:
  allocate 2              % 2 local variables: Y1, Y2
  get_variable X3, A1     % load 'K' to X3.
  get_variable Y1, A2     % load 'L' to Y1.
```

Compilation of Facts, Rules and Queries

The code for a rule $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots).$
first checks if the arguments passed to it (in $A_1, \ldots, A_n$) match $t_1, \ldots, t_n$.
Then, $p_1, \ldots, p_m$ are called; for each call, the arguments of the respective p_i are loaded into $A_1, \ldots$. Facts are the special case with $m = 0$ and queries the special case with $n = 0$.

Local variables and arguments which are not only used in p_1 have to be saved in the stack (in $Y_1, \ldots$) for later use.

Example: `path(K,L) :- edge(K,M), path(M,L).`

```
path:
  allocate 2              % 2 local variables: Y1, Y2
  get_variable X3, A1     % load 'K' to X3.
  get_variable Y1, A2     % load 'L' to Y1.
  put_value    X3, A1     % pass 'K' and
  put_variable Y2, A2     % 'M' (new var in Y2) to
  call edge               % edge.
```

Compilation of Facts, Rules and Queries

The code for a rule $r(t_1, \ldots, t_n) \ :- \ p_1(\ldots), \ldots, p_m(\ldots).$
first checks if the arguments passed to it (in $A_1, \ldots, A_n$) match $t_1, \ldots, t_n$.
Then, $p_1, \ldots, p_m$ are called; for each call, the arguments of the respective p_i are loaded into $A_1, \ldots$. Facts are the special case with $m = 0$ and queries the special case with $n = 0$.

Local variables and arguments which are not only used in p_1 have to be saved in the stack (in $Y_1, \ldots$) for later use.

Example: `path(K,L) :- edge(K,M), path(M,L).`

```
path:
  allocate 2              % 2 local variables: Y1, Y2
  get_variable X3, A1     % load 'K' to X3.
  get_variable Y1, A2     % load 'L' to Y1.
  put_value    X3, A1     % pass 'K' and
  put_variable Y2, A2     % 'M' (new var in Y2) to
  call edge               % edge.
  put_value    Y2, A1     % pass 'M' and
  put_value    Y1, A2     % 'L' to
  call path               % path.
  deallocate              % clean up and call continuation.
```

Construction of Terms

When compound terms occur on a right-hand side, they have to be constructed on the heap.

Example: `?- add(A,B,succ(succ(A)))`

```
query:
  set_variable   A1          % create new variable 'A'.
  set_variable   A2          % create new variable 'B'.
  put_structure  succ,X4     % succ(...) onto heap, ref. in X4.
  set_value      A1          % 'A' as argument of succ.
  put_structure  succ,A3     % succ(...) onto heap, ref. in A3.
  set_value      X4          % succ(A) as argument of succ.
  call add                   % call add.
  proceed                    % call continuation.
```

Checking Arguments

Checking arguments is done using `get_structure` to test the constructor of a term and `unify_variable` or `unify_value` for the unification of (sub-)terms.

Example: `add(succ(A),B,succ(C)) :- add(A,B,C).`

```
add:
  get_structure  succ,A1      % does A1 start with succ(...)?.
  unify_variable X4           % unify argument of succ with new var in X4
  get_variable   X5,A2        % load 'B' to X5.
  get_structure  succ,A3      % does A3 start with succ(...)?
  unify_variable X6           % unify argument of succ with new var in X6.
  put_value      X4,A1        % pass 'A',
  put_value      X5,A2        % 'B' and
  put_value      X6,A3        % 'C'
  call add                    % to add.
  proceed
```

Modes

With `get_structure` f,A_i, there can be two principal situations:

- A_i is an unbound variable $\rightsquigarrow$ A_i is bound to the newly created constructor f in the heap, register S is set to the succeeding heap address and `unify_variable`/`unify_value` *write* their respective argument at address S (and increment S afterwards).
- A_i is already bound to a term $\rightsquigarrow$ A_i must point to a constructor f (otherwise: *fail*); S is set to the first argument of f, `unify_variable`/`unify_value` *read* from S, `unify_value` additionally unifies its argument with the value read (and both increment S afterwards).

`get_structure` switches between the *read mode* and *write mode*. Depending on the mode, `unify_variable` and `unify_value` have different behavior.

Necessary layout of terms in memory: the arguments of terms must directly follow the term itself.

Backtracking on Alternatives

`get_structure`, `get_value` and `unify_value` can fail (when the unification does not succeed). In this case, *backtracking* is performed, i.e, a jump to the next alternative. Backtracking is managed with

- `try_me_else` *L*:
  initialize the trail for (new) variable bindings, *L* is set as return address for backtracking
- `retry_me_else` *L*:
  reset variable bindings, *L* is set as new return address for backtracking
- `trust_me`:
  reset variable bindings, finish trail management, i.e., deallocate the current entry on the trail

Managing trail and backtracking is performed using registers that are set by the above instructions and the values of which are stored intermittently on the stack.

WAM Code for `path`

(Assuming that `path` consists of 3 rules.)

```
path:
  try_me_else path2        - path(X,Y) :- edge(X,Y).
  call edge                - forward arguments ('X' und 'Y') to edge
  proceed
path2:
  retry_me_else path3
  ...
path3:
  trust_me                 - path(X,Y) :- edge(X,Z), path(Z,Y).
  allocate 2               - 2 locale variables: Y1, Y2
  get_variable X3, A1      - load 'X' to X3.
  get_variable Y1, A2      - load 'Y' to Y1.
  put_value    X3, A1      - pass 'X' and
  put_variable Y2, A2      - and 'Z' (new var in Y2)
  call edge                - to edge.
  put_value    Y2, A1      - pass 'Z'
  put_value    Y1, A2      - and 'Y'
  call path                - to path.
  deallocate
```

There are More Details...

The presentation here is a sketch of the WAM. Several aspects are not discussed, in particular, the details of memory allocation and unification. For example, the WAM imposes the following rules:

- When two variables are unified, the variable with the higher address is made to refer to variable at the lower address.
- The stack has to have higher addresses than the heap. $\rightsquigarrow$ Variables in the heap *never* reference variables in the stack (to avoid dangling references in the heap).
- Free variables on a right-hand side, for example, `Y` in
  ```
  a(X) :- b(X,Y),c(Y,X).
  ```
  must not be allocated on the stack (to enable stack optimizations).

Optimizations

We have considered 17 instructions of the Warren Abstract Machine here. These suffice to implement Prolog[2]. The Warren Abstract Machine has 39 instructions in total; the further instructions serve the following purposes:

- implementation of the *cut* (3),
- efficient handling of constants (4) and lists (4), and for the
- efficient handling of special situations (11).

For more details have a look at:
Hassan Aït-Kaci, “Warren’s Abstract Machine: A Tutorial Reconstruction”, MIT Press, 1991, online: `https://github.com/a-yiorgos/wambook`

[2] without *cut*